TELEFÓNICA DEL PERÚ — RED DE ACCESO ÓPTICO

UNIVERSIDAD TECNOLÓGICA DEL PERÚ | FACULTAD DE INGENIERÍA DE SISTEMAS

DOCUMENTO TÉCNICO OFICIAL — ENTREGABLE 4.3

ANEXO A: DOCUMENTACIÓN TÉCNICA DEL PROYECTO MEDIANTE MARKDOWN

Guía de Arquitectura, Despliegue DevOps, Catálogo de API RESTful y Protocolos de Operación SRE

Sistema: Inventario Digital de ODF (ODF Tracker)

Asignatura: Curso Integrador I: Sistemas Software

Docente: Ing. Rudy Chapoñan Camarena

Integrantes: Rodrigo Layme Ureta (U22244493)
Kevin Smith Cusi Espinoza (U23220459)
Diana Greis Aguado Pacompia (U23210141)

Versión de Código: v1.0.0 (Release Sprint 2 — TDD 100%)

Fecha de Emisión: Octubre de 2026

Lima — Perú, 2026

1. Visión General del Sistema y Ficha Técnica

El presente documento constituye el **Anexo Técnico A** del proyecto *Sistema de Inventario Digital de ODF* para Telefónica del Perú. Ha sido diseñado bajo el estándar de documentación técnica viva mediante herramientas como **Markdown**, garantizando que el personal de ingeniería, desarrollo, DevOps y soporte de red cuente con una referencia precisa, unívoca y ejecutable para el ciclo de vida del software.

> **Propósito de la Solución**
> Digitalizar y gobernar la asignación física de puertos ópticos en distribuidores de fibra (ODF), eliminando registros manuales en hojas de cálculo y previniendo colisiones de hilos ópticos mediante transacciones ACID, políticas de acceso granular (RBAC) y validación criptográfica anti-CSRF.

1.1. Matriz de Compatibilidad y Requisitos de Infraestructura

El sistema se encuentra homologado para entornos locales de desarrollo (XAMPP / Windows) y servidores de producción Linux (Ubuntu Server 20.04 LTS / 22.04 LTS o RHEL 8):

Componente	Versión Mínima	Finalidad y Extensiones Requeridas
Sistema Operativo	Linux x64 / Windows 10/11 Server	Hospedaje del servidor web y servicios de backend.
Servidor Web HTTP	Apache 2.4+ (o Nginx 1.18+)	Servidor web con motor de reescritura `mod_rewrite` y soporte `.htaccess`.
Intérprete Backend	PHP 8.1.0 o superior (64-bit)	Tipado estricto con extensiones: `pdo_mysql`, `mbstring`, `json`, `openssl`, `zip`, `session`.
Motor de Base de Datos	MySQL 8.0+ / MariaDB 10.4+	Almacenamiento transaccional (InnoDB), juego de caracteres `utf8mb4` e integridad referencial 3FN.
Gestor de Dependencias	Composer 2.2+	Carga automática de clases bajo estándar PSR-4 y gestión de librerías de soporte.
Suite de Calidad TDD	PHPUnit 10.0+ / 11.0+	Ejecución de pruebas unitarias automatizadas (32 pruebas, 128 aserciones activas).

2. Guía Paso a Paso de Despliegue e Instalación (DevOps)

A continuación se detalla el procedimiento de aprovisionamiento en un servidor limpio para garantizar la replicabilidad del entorno:

Paso 1: Clonación del Repositorio de Código Fuente

Obtener el código fuente desde el repositorio institucional protegido:

```
git clone https://github.com/telefonica-peru/inventario_odf.git
cd inventario_odf
git checkout main
```

Paso 2: Instalación de Dependencias PSR-4 mediante Composer

Instalar las dependencias de producción optimizando el autoloader de clases:

```
composer install --no-dev --optimize-autoloader
```

Paso 3: Parametrización del Archivo de Entorno Confidencial (.env)

Copiar la plantilla de entorno seguro y configurar las credenciales de conexión:

```
cp .env.example .env
# Configuración en .env:
APP_ENV=production
APP_DEBUG=false
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=inventario_odf
DB_USERNAME=odf_user
DB_PASSWORD=C0ntras3naSegura2026!$
SESSION_LIFETIME=7200
```

> **Regla de Seguridad Crítica**
>
> El archivo `.env` contiene secretos criptográficos y credenciales de base de datos. Se encuentra bloqueado a través de directivas en `public/.htaccess` para evitar cualquier exposición web inadvertida.

Paso 4: Aprovisionamiento de la Base de Datos Relacional

Ejecutar la creación de tablas, triggers de auditoría forense y carga de catálogos semilla:

```
mysql -u odf_user -p inventario_odf < sql/schema.sql
mysql -u odf_user -p inventario_odf < sql/seed_data.sql
```

Paso 5: Configuración de Permisos y Virtual Host de Apache

Apuntar el `DocumentRoot` exclusivamente al directorio `public/` y otorgar permisos de escritura en la carpeta de bitácoras:

```
# En Linux:
chown -R www-data:www-data storage/logs/
chmod -R 775 storage/logs/

# Directiva Apache VirtualHost:
<VirtualHost *:80>
    ServerName odf.telefonica.com.pe
    DocumentRoot "C:/xampp/htdocs/inventario_odf/public"
    <Directory "C:/xampp/htdocs/inventario_odf/public">
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

3. Arquitectura de Directorios y Estructura de Carpetas

El código fuente se estructura desacoplando la lógica de negocio del acceso web público:

```
inventario_odf/
├── app/                        # Capa de Lógica de Negocio y Dominio (PSR-4)
│   ├── Config/                 # Singleton Database, Csrf, EnvLoader
│   ├── Contracts/Daos/         # Interfaces de Abstracción DAO (DIP)
│   ├── Daos/                   # Implementaciones MySQL con transacciones ACID
│   ├── Entities/               # Clases de Dominio (Puerto, Odf, Site, Cable, Usuario)
│   ├── Enums/                  # Enumeraciones tipadas de estado y roles
│   ├── Policies/               # Patrón Factory y Políticas de Autorización RBAC
│   ├── Services/               # Servicios de Dominio (BatchImport, Stats, Infra)
│   └── bootstrap.php           # Arranque de sesión segura y autoloader
├── config/                     # Archivos de configuración estática del sistema
├── public/                     # Raíz pública accesible por el servidor web (Front Controller)
│   ├── api/                    # Endpoints RESTful asíncronos (AJAX / JSON)
│   │   ├── auth_login.php      # Autenticación con BCrypt y Rate Limiting
│   │   ├── auth_logout.php     # Destrucción segura de sesión
│   │   ├── batch_import_puertos.php # Ingesta masiva CSV/XLSX
│   │   ├── get_puertos.php     # Consulta reactiva de puertos por ODF
│   │   ├── health.php          # Health Check para observabilidad SRE
│   │   └── update_puerto.php   # Modificación atómica de puertos y auditoría
│   ├── assets/                 # Hojas de estilo CSS, scripts JS reactivos, iconos
│   ├── plantillas/             # Plantillas descargables para carga masiva
│   ├── .htaccess               # Cabeceras defensivas y redirecciones
│   └── index.php               # Front Controller principal de la aplicación
├── sql/                        # Scripts DDL de base de datos y semillas
├── storage/logs/               # Bitácora rotativa de auditoría y errores del sistema
├── tests/                      # Suite de pruebas automatizadas PHPUnit (32 tests)
├── .env                        # Variables confidenciales de producción (protegido)
├── .env.example                # Plantilla de variables de entorno para despliegue
├── composer.json               # Definición de dependencias y mapeo de namespaces
└── phpunit.xml                 # Configuración de pruebas unitarias
```

4. Catálogo y Especificación de Endpoints de la API RESTful

Todos los servicios internos emplean el protocolo HTTP/1.1 con intercambio de información en formato JSON estándar:

Endpoint	Método	Autenticación	Payload Requerido	Códigos y Respuestas
`/api/auth_login.php`	POST	Pública CSRF REQ	`{ "email": "str", "password": "str", "csrf_token": "str" }`	**200 OK:** Redirección y sesión **401:** Credenciales inválidas **429:** Rate Limit excedido
`/api/auth_logout.php`	POST	Sesión activa	`{ "csrf_token": "str" }`	**200 OK:** Sesión invalidada **403:** Token CSRF ausente
`/api/get_puertos.php`	GET	Sesión activa	Query params: `?odf_id=INT&bandeja=INT`	**200 OK:** Array de entidades Puerto **404:** ODF inexistente
`/api/update_puerto.php`	POST	Sesión activa RBAC: EDIT	`{ "puerto_id": INT, "estado": "str", "cable_id": INT, "fibra": INT, "csrf_token": "str" }`	**200 OK:** Transacción exitosa **403:** Sin privilegio RBAC **422:** Colisión de hilo óptico
`/api/batch_import_puertos.php`	POST	Sesión activa RBAC: ADMIN	Multipart: `file (CSV/XLSX)`, `odf_id`, `csrf_token`	**200 OK:** Total importados e incidencias **400:** Cabeceras corruptas
`/api/health.php`	GET	Monitoreo SRE	Ninguno	**200 OK:** Diagnóstico satisfactorio **503:** Falla crítica de componente

5. Protocolo de Diagnóstico y Salud Operativa (Health Check / SRE)

Para garantizar la observabilidad del sistema en centros de operaciones de red (NOC), se dispone del endpoint `/api/health.php`, el cual evalúa de forma sintética los componentes críticos sin exponer información sensible:

- **Ping de Base de Datos:** Ejecuta una consulta ligera transaccional (`SELECT 1`) confirmando la disponibilidad del motor MySQL y latencia de conexión.
- **Verificación de Almacenamiento:** Comprueba permisos de lectura y escritura en el directorio `storage/logs/`.
- **Sanitización de Fallas:** Ante caídas de la base de datos o fallas de I/O, emite un código HTTP `503 Service Unavailable` suprimiendo volcados de memoria y trazas de servidor para evitar ataques de reconocimiento (CWE-209).

6. Registro de Auditoría de Seguridad y Refactorizaciones Críticas

La siguiente bitácora documenta las vulnerabilidades remediadas durante la auditoría de seguridad defensiva previa al pase a producción:

Vulnerabilidad Identificada	Causa Raíz	Acción Correctiva / Refactorización	Severidad
Bypass de Rate Limiting en Login	Contador de intentos fallidos residía en `$_SESSION`, reseteable al eliminar la cookie de sesión.	Implementación de persistencia compartida en archivo con control de IP y correo en `public/api/auth_login.php`.	**ALTA (MITIGADA)**
Condición de Carrera en Sesiones	Llamados asíncronos simultáneos sin parámetros estrictos de cookie.	Configuración unificada en `app/bootstrap.php` con `use_strict_mode`, `HttpOnly` y `SameSite=Strict`.	**MEDIA (MITIGADA)**
Ausencia de Cabeceras Defensivas	Apache entregaba respuestas sin protección contra MIME-sniffing y Clickjacking.	Inyección de cabeceras en `public/.htaccess` (`X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`).	**MEDIA (MITIGADA)**
Divulgación de Host en Health Check	Trazas de error PDO revelaban nombres de host internos en respuestas 500.	Sanitización en `public/api/health.php` emitiendo únicamente mensaje higienizado de diagnóstico.	**BAJA (MITIGADA)**

7. Diagrama de Arquitectura Multicapa MVC y Capas DAO

El siguiente diagrama técnico sintetiza el flujo de procesamiento de peticiones, capas de abstracción y persistencia transaccional:

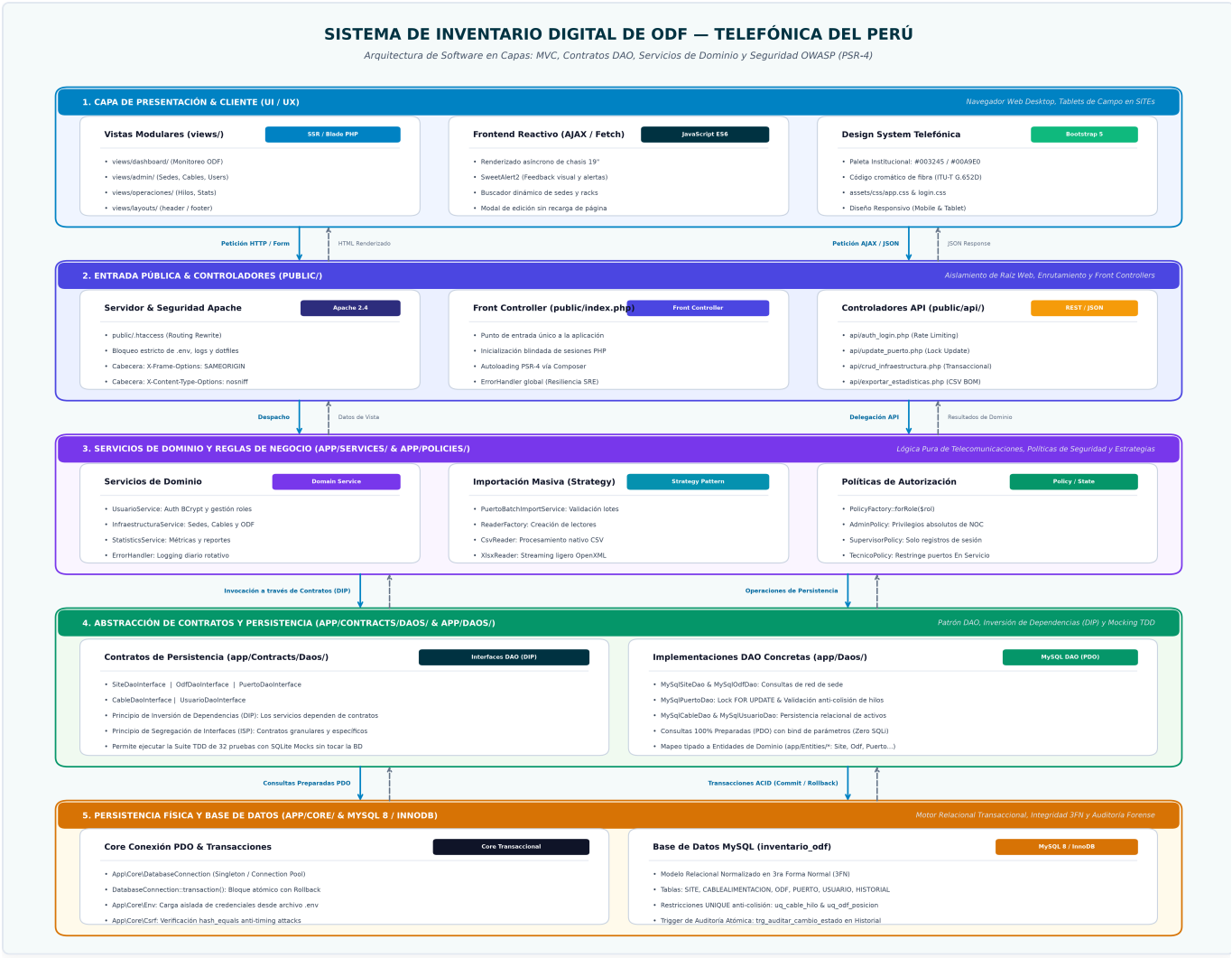


Figura A.1: Diagrama de Arquitectura Multicapa MVC, Front Controller y Capas DAO del Sistema ODF Tracker